

Decision Tree Classifier

Decision Tree Classifier is a **supervised machine learning algorithm** used for **classification**. It:

- Splits data based on feature values
- Uses an impurity metric (Gini / Entropy)
- Creates rule-based decisions (flowchart style)
- Ends with class labels at leaf nodes

Classification Types

- **Binary classification:** 2 classes
- **Multiclass classification:** 3+ classes
- **Multi-output classification:** multiple labels predicted together

Tree Components

- **Root node:** first decision node
- **Internal node:** split using a feature condition
- **Leaf node:** final predicted class
- **Depth:** number of levels in tree
- **Impurity:** measure of mixed classes in a node

Internal Working of Decision Tree

- Start with full dataset at root node
- Calculate impurity at current node
- Try possible splits on each feature
- Compute impurity after each split (weighted)
- Select split producing best improvement
- Repeat recursively for left and right child nodes
- Stop when stopping condition is satisfied

Stopping Conditions

- Node becomes pure (all samples same class)

- `max_depth` reached
- `min_samples_split` condition not satisfied
- `min_samples_leaf` condition not satisfied
- No useful impurity decrease

Impurity Metrics and Formulas

Class probability

$$p_k = \text{count}_k / N$$

Gini Impurity

$$\text{Gini}(S) = 1 - \sum (p_k^2)$$

Binary shortcut

$$\text{Gini} = 2 * p * (1 - p)$$

Range

- Minimum: 0 (pure node)
- Maximum for binary: 0.5 (most impure)

Entropy

$$\text{Entropy}(S) = - \sum (p_k * \log_2(p_k))$$

Weighted impurity after split

$$\text{Impurity_after} = \sum ((|S_i| / |S|) * \text{Impurity}(S_i))$$

Gini Reduction

$$\text{GiniReduction} = \text{Gini}(S) - \sum ((|S_i| / |S|) * \text{Gini}(S_i))$$

Information Gain

$$\text{IG} = \text{Entropy}(S) - \sum ((|S_i| / |S|) * \text{Entropy}(S_i))$$

Ball Classification Case Study

Encoding Rules

- Tennis = 1
- Cricket = 2
- Rough = 1

- Smooth = 0

Domain Rule

- Cricket weight: **90–110 grams**, **Smooth** surface
- Tennis weight: **30–70 grams**, **Rough** surface

Dataset (10 samples)

Features: [Weight, Surface]

Labels: **BallType**

Tennis (1):

[35,1], [40,1], [55,1], [65,1], [70,1]

Cricket (2):

[92,0], [95,0], [100,0], [105,0], [110,0]

Root Node Calculation (Gini)

At root: 10 samples

- Tennis = 5
- Cricket = 5

$$P(\text{Tennis}) = 5/10 = 0.5$$

$$P(\text{Cricket}) = 5/10 = 0.5$$

$$\text{Gini} = 1 - (0.5^2 + 0.5^2)$$

$$\text{Gini} = 1 - (0.25 + 0.25)$$

$$\text{Gini} = 1 - 0.5$$

$$\text{Gini} = 0.5$$

Impure node.

Split Trial on Weight

Choose a threshold: $\text{Weight} \leq 80$

Left node (≤ 80)

35, 40, 55, 65, 70 → all Tennis

$$P(\text{Tennis}) = 1$$

$$\text{Gini}_{\text{left}} = 1 - 1^2 = 0$$

Right node (> 80)

92, 95, 100, 105, 110 → all Cricket

$$P(\text{Cricket}) = 1$$

$$\text{Gini}_{\text{right}} = 1 - 1^2 = 0$$

Weighted Gini after split

$Gini_after = (5/10)*0 + (5/10)*0$

$Gini_after = 0$

Perfect split.

Final Tree Structure

```
Weight <= 80 ?  
/  \  
Yes No  
(Tennis=1) (Cricket=2)
```

Tree depth = 1

Surface not selected because:

- Weight split already gives impurity = 0
- Children become pure leaf nodes
- Tree stops immediately

Important Parameters of DecisionTreeClassifier

```
DecisionTreeClassifier(  
    criterion='gini',  
    splitter='best',  
    max_depth=None,  
    min_samples_split=2,  
    min_samples_leaf=1,  
    max_features=None,  
    random_state=None,  
    class_weight=None,  
    ccp_alpha=0.0  
)
```

criterion

- 'gini': fast, default
- 'entropy': information gain based
- 'log_loss': probabilistic measure

splitter

- 'best': best split (default)

- 'random': random split among candidates

max_depth

- Limits depth
- Controls complexity
- Prevents overfitting

min_samples_split

- Minimum samples required to split a node
- Larger value reduces tree growth

min_samples_leaf

- Minimum samples required at leaf
- Avoids tiny leaves and overfitting

max_features

- Limits number of features checked per split
- Useful when features are many

random_state

- Fixes randomness
- Helps reproducible output

class_weight

- Handles imbalanced dataset
- 'balanced' works well for rare classes

ccp_alpha

- Post-pruning strength
- Higher value => simpler tree

Python Code

```
from sklearn.tree import DecisionTreeClassifier
```

```
# Features: [Weight, Surface] Surface: Rough=1, Smooth=0  
X = [  
    [35,1], [40,1], [55,1], [65,1], [70,1],
```

```

    [92,0], [95,0], [100,0], [105,0], [110,0]
]

# Labels: Tennis=1, Cricket=2
y = [1,1,1,1,1, 2,2,2,2,2]

# Model
model = DecisionTreeClassifier(criterion="gini",
random_state=42) model.fit(X, y)

# Test sample
test_ball = [85, 0] # weight=85, smooth
pred = model.predict([test_ball])[0]

print("Prediction:", "Tennis" if pred == 1 else "Cricket")

from sklearn.tree import plot_tree
import matplotlib.pyplot as plt

plt.figure(figsize=(7,5))
plot_tree(model,
    feature_names=["Weight", "Surface"],
    class_names=["Tennis", "Cricket"],
    filled=True)
plt.show()

```